QUIZ PRE-UTS

Natural Language Processing

NLP36013 • UIN Siber syekh Nurjati Cirebon

Abdul Kharis | NIM: 2388010004 | Informatika 6A | UIN Siber Syekh Nurjati Cirebon

Petunjuk Pengerjaan

- Sertakan rumus, langkah perhitungan, dan analisis yang mendukung jawaban. - Tidak diperkenankan membuka AI. Hanya di perbolehkan membuka catatan.

1. Text Preprocessing

```
" Produk ini SANGAT bagus! ! Saya sudah beli 3x dan ga nyesel : ) "
```

Terapkan minimal 4 tahapan text preprocessing yang relevan pada kalimat di atas. Untuk setiap tahapan, jelaskan: (a) nama tahapan, (b) tujuannya, dan (c) hasil transformasinya. Kemudian diskusikan: apakah semua tahapan selalu wajib diterapkan dalam setiap tugas NLP? Berikan argumen dengan contoh.

Tahapan	Tujuan	Hasil Transformasi
Case Folding	Menyamakan huruf besar dan kecil agar kata seperti 'SANGAT' dan 'sangat' dianggap sama. Ini mencegah duplikasi fitur di vocabulary.	produk ini sangat bagus!! saya sudah beli 3x dan ga nyesel :)
Cleaning (Noise Removal)	Menghapus karakter yang tidak punya nilai semantik (tanda baca berulang, emoji, simbol). Karakter ini hanya menambah noise tanpa makna.	produk ini sangat bagus saya sudah beli 3x dan ga nyesel
Tokenisasi	Memecah kalimat menjadi unit terkecil (token/kata). Model NLP bekerja pada level token, bukan kalimat utuh.	['produk', 'ini', 'sangat', 'bagus', 'saya', 'sudah', 'beli', '3x', 'dan', 'ga', 'nyesel']
Stopword Removal	Menghapus kata umum yang sering muncul tapi tidak penting secara semantik (seperti 'ini', 'dan'). Ini mengurangi dimensi fitur.	['produk', 'sangat', 'bagus', 'beli', '3x', 'ga', 'nyesel']

Normalisasi & Stemming	Mengubah kata informal/slang ke bentuk baku dan mengurangi kata berimbuhan ke bentuk dasar. Contoh: 'ga' → 'tidak', 'nyesel' → 'sesal'.	['produk', 'sangat', 'bagus', 'beli', 'tidak', 'sesal']

Kalimat awal: "Produk ini SANGAT bagus!! Saya sudah beli 3x dan ga nyesel :)"
Hasil akhir: ['produk', 'sangat', 'bagus', 'beli', 'tidak', 'sesal']

Diskusi: Apakah Semua Tahapan Wajib?

Jawaban: Tidak selalu.

penerapan tahapan preprocessing itu bergantung pada konteks tugas dan karakteristik data. Tidak ada pendekatan *one for all* atau *one-size-fits-all*.

Argumen dan contoh kasus:

- Case Folding
  Hampir selalu diperlukan untuk klasifikasi teks umum. Tapi, jika tugasnya huruf kapital itu penting.

  Contoh: kata "Apple" (perusahaan) beda dengan "apple" (buah). Jika difolding lowercase, model tidak sesuai.

- Noise Removal
  Penting untuk analisis sentimen media sosial untuk emoji juga tanda seru tidak dianggap noise. Tapi tergantung tujuan beberapa kasus memungkinkan emoji dianggap fitur penting.

  Contoh: untuk analisis sentimen, emoji menjadi indikator kuat emosi positif yang tidak boleh dibuang.

- Stopword Removal
  Umumnya berguna untuk mengurangi dimensi. Tapi, di analisis sentimen, kata seperti "tidak", "bukan", atau "kurang" bisa menjafi penting karena mengubah polaritas kalimat. Jika dihapus sebagai stopword, makna "tidak bagus" jadi "bagus".

- Stemming
  Sangat membantu untuk Bahasa Indonesia yang kaya imbuhan (me-, ber-, -kan). Tapi, Tapi stemmer-nya kurang akurat (over-stemming), makna bisa rusak.

Contoh: kata "mengelola" dan "pengelola" distem jadi "kelola", yang mungkin kehilangan nuansa konteks tertentu.

Kesimpulan:
Preprocessing itu disesuaikan dengan kebutuhan model.

2. Bag-of-Words & Document-Term Matrix Diberikan dua dokumen:

```
DI • ″ saya suka makan nasi goreng″
D2• ″ saya tidak suka nasi padang″
```

- Buatlah Document-Term Matrix (DTM) dari kedua dokumen. Tuliskan vocabulary dan nilai frekuensi tiap term.
- Sebutkan dan jelaskan tiga keterbatasan utama representasi BOW yang dapat menurunkan performa model klasifikasi teks.

No	Kata (Vocabulary)
1	saya
2	suka
3	makan
4	nasi
5	goreng
6	tidak
7	padang

Total vocabulary |V| = 7 kata unik

DTM

Dokumen	saya	suka	makan	nasi	goreng	tidak	padang
D1	1	1	1	1	1	0	0
D2	1	1	0	1	0	1	1

Representasi vektor:

```
D1 = [1, 1, 1, 1, 1, 0, 0]
D2 = [1, 1, 0, 1, 0, 1, 1]
```

Tiga Keterbatasan Utama BoW

- Urutan kata nggak diperhatiin
  Kalimat "saya tidak suka" dan "saya suka tidak" bakal direpresentasi sama, padahal artinya beda jauh. Ini masalah besar buat tugas yang butuh konteks.
- Nggak nangkep makna semantik
  Kata "bagus", "keren", "mantap" dianggap fitur beda, padahal maksudnya mirip. Model jadi nggak bisa generalisasi dengan baik.
- Vektor jadi sparse dan dimensi membengkak
  Kalau vocabulary ribuan, tiap dokumen cuma pakai sebagian kecil. Hasilnya matriks kebanyakan nol → boros memori, komputasi berat, dan risiko overfitting.

3. TF-IDF & Smooth IDF

Diberikan kondisi berikut :

```
Korpus      5 dokumen
Kata        ” indonesia ” muncul di
Frekuensi   2 dari 5 dokumen
Pada Dl     100 kata total, kata ” indonesia” muncul 4 kali
```

- Tuliskan formula TF, IDF standar, dan Smooth IDF. Jelaskan perbedaan motivasinya.
- Hitung TF, IDF Standar, Smooth IDF, TF-IDF Standar, dan TF-IDF Smooth untuk kata ”indonesia” pada Dl . Tampilkan langkah-langkah perhitungan.
- Apa masalah yang terjadi pada IDF standar jika sebuah kata muncul di semua dokumen? Bagaimana Smooth IDF mengatasinya?

Formula

Metrik	Formula	Motivasi
TF	$TF(t,d) = \frac{f_{t,d}}{\sum_k f_{k,d}}$	Normalisasi frekuensi kata agar dokumen panjang tidak otomatis dominan
IDF Standar	$IDF(t) = \log\left(\frac{N}{df_t}\right)$	Beri bobot tinggi pada kata langka; kata yang muncul di banyak dokumen dianggap kurang informatif
Smooth IDF	$IDF_{smooth}(t) = \log\left(\frac{N+1}{df_t+1}\right) + 1$	Hindari $IDF = 0$ saat kata muncul di semua dokumen, dan cegah division by zero saat $df_t = 0$

- IDF Standar: Asumsi kata yang ada di semua dokumen itu "noise" → bobot nol.
- Smooth IDF: Lebih konservatif; tetap kasih bobot minimal positif biar kata universal nggak hilang sepenuhnya, plus lebih aman buat data baru.

Hitung TF

$$TF(\text{'indonesia'}, D1) = \frac{4}{100} = 0.04$$

Hitung IDF Standar

$$N = 5, \quad df(\text{'indonesia'}) = 2$$

Hitung Smooth IDF

$$IDF_{\text{standar}} = \log\left(\frac{5}{2}\right) = \log(2.5) \approx 0.9163 \quad (\text{basis } e)$$

Hitung TF-IDF

$$IDF_{smooth} = \log\left(\frac{N+1}{df_t+1}\right) + 1 = \log\left(\frac{6}{3}\right) + 1 = \log(2) + 1 \approx 0.6931 + 1 = 1.6931$$

$$TF\text{-}IDF_{standar} = 0.04 \times 0.9163 = 0.03665, \quad TF\text{-}IDF_{smooth} = 0.04 \times 1.6931 = 0.06772$$

Masalah IDF Standar:

- IDF = 0 jika kata muncul di semua dokumen → log(N/N) = log(1) = 0 → kata relevan tapi umum jadi diabaikan total.
- Division by zero jika kata baru muncul saat inferensi (df(t) = 0) → rumus jadi tidak terdefinisi.

Solusi Smooth IDF:

$$IDF_{smooth}(t) = \log\left(\frac{N+1}{df_t+1}\right) + 1$$

- +1 di pembilang & penyebut → hindari pembagian nol dan IDF = 0.
- +1 di luar log → pastikan bobot selalu positif, walau kata muncul di semua dokumen.

Smooth IDF lebih robust karena tetap memberi bobot minimal pada kata universal dan aman untuk kata baru.

4. Cosine Similarity

Diberikan dua vektor TF-IDF dokumen:

```
DI = [0.5, 0.0, 0.3, 0.8]
D2 = [0.4, 0.6, 0.3, 0.0]
```

- Hitung cosine similarity antara DI dan D2. Tunjukkan seluruh langkah: dot product, magnitude masing-masing, dan hasil akhir.
- Mengapa cosine similarity lebih tepat dibanding Euclidean distance untuk teks? Hubungkan jawaban dengan karakteristik vektor TF-IDF.

D1 = [0.5, 0.0, 0.3, 0.8]

D2 = [0.4, 0.6, 0.3, 0.0]

Dot Product ($D1 \cdot D2$)

$= (0.5\times0.4) + (0.0\times0.6) + (0.3\times0.3) + (0.8\times0.0)$

$= 0.20 + 0 + 0.09 + 0$

$= 0.29$

Magnitude Masing-masing

$||D1|| = \sqrt{(0.5^2 + 0.0^2 + 0.3^2 + 0.8^2)}$

$= \sqrt{(0.25 + 0 + 0.09 + 0.64)}$

$= \sqrt{0.98} \approx 0.9899$

$||D2|| = \sqrt{(0.4^2 + 0.6^2 + 0.3^2 + 0.0^2)}$

$= \sqrt{(0.16 + 0.36 + 0.09 + 0)}$

$= \sqrt{0.61} \approx 0.7810$

Cosine Similarity

$\cos(\theta) = 0.29 / (0.9899 \times 0.7810)$

$= 0.29 / 0.7732$

≈ 0.375

Kenapa Cosine Lebih Cocok untuk Teks?

Karena vektor TF-IDF itu sparse dan dimensinya tinggi. Euclidean distance sensitif sama magnitude — dokumen panjang bakal kelihatan "jauh" meski topiknya sama. Cosine similarity cuma liat arah vektor, jadi fokus ke pola distribusi kata, bukan seberapa panjang dokumennya. Ini lebih cocok buat ukur kemiripan topik.

5. Naive Bayes dengan Laplace Smoothing Data latih sentimen:

```
Kelas POSITIF (10 dokumen) kata = 50      "bagus" = 8x,    "jelek" = 2x,    total kata
Kelas NEGATIF (10 dokumen) kata = 50      "bagus" = 3x,    "jelek" = 9x,    total kata
Vocabulary : 100 kata unik
P (Positif) = P (Negatif) = 0.5
```

Klasifikasikan kalimat "produk bagus" menggunakan Naive Bayes dengan Laplace Smoothing (k=l). Tunjukkan perhitungan P(kelas I kalimat) untuk kedua kelas secara lengkap, bandingkan nilainya, dan berikan kesimpulan akhir klasifikasi.

```
Rumus: P(w|c) = (count(w,c)+1) / (total_kata_c + V)

Kata "produk" (nggak ada di training):
P("produk"|Pos) = (0+1)/(50+100) = 1/150 ≈ 0.0067
P("produk"|Neg) = sama = 0.0067

Kata "bagus":
P("bagus"|Pos) = (8+1)/150 = 9/150 = 0.06
P("bagus"|Neg) = (3+1)/150 = 4/150 ≈ 0.0267

Posterior (pakai log biar nggak underflow):
log P(Pos|D) = log(0.5) + log(0.0067) + log(0.06)
             ≈ -0.6931 + (-5.0106) + (-2.8134) = -8.5171

log P(Neg|D) = log(0.5) + log(0.0067) + log(0.0267)
             ≈ -0.6931 + (-5.0106) + (-3.6243) = -9.3280

-8.5171 > -9.3280 → POSITIF
```

Kalimat "produk bagus" diklasifikasikan POSITIF, didorong terutama oleh kata "bagus" yang lebih sering muncul di kelas positif.

6. Logistic Regression Multiclass
   - Jelaskan perbedaan mendasar antara pendekatan One-vs-Rest (OvR) dan Softmax (Multinomial) Logistic Regression, dari segi mekanisme dan output probabilitasnya.
   - Berikan skenario nyata di mana Softmax lebih tepat daripada OvR. Jelaskan alasannya.
   - Apa kelemahan OvR ketika jumlah kelas sangat banyak (misal 50 kelas) dan data tidak seimbang?

Aspek	One-vs-Rest (OvR)	Softmax
Mekanisme	Latih K classifier biner (satu kelas vs sisa)	Satu model langsung output probabilitas semua kelas
Output	Probabilitas tiap kelas independen, nggak harus jumlah=1	Probabilitas dinormalisasi → jumlah semua kelas = 1
Asumsi	Kelas dianggap independen	Kelas saling eksklusif (mutually exclusive)

Kapan Softmax Lebih Tepat?
Contoh: Klasifikasi topik berita (Politik/Ekonomi/Olahraga/Teknologi). Satu artikel cuma boleh masuk satu kategori. Softmax memastikan probabilitas "bersaing" — kalau Politik naik, yang lain turun. Lebih intuitif dan konsisten.

Kelemahan OvR Kalau Kelas Banyak + Imbalanced
Harus latih banyak classifier (misal 50) → berat komputasi
Tiap classifier hadapi imbalance ekstrem (1 vs 49) → cenderung bias ke kelas mayoritas
Output probabilitas nggak terkalibrasi → susah bandingin kepercayaan antar kelas

7. Data Leakage

Seorang mahasiswa membangun model klasifikasi teks dengan langkah berikut:

```
Langkah 1. Load seluruh dataset (train + test)
Langkah 2. Fit TF-IDF vectorizer pada SELURUH dataset
Langkah 3. Transform seluruh dataset menggunakan vectorizer tersebut
Langkah 4. Split data menjadi train set dan test set
Langkah 5. Latih model pada train set
Langkah 6. Evaluasi model pada test set
```

- Identifikasi dengan tepat di langkah berapa data leakage terjadi dan jelaskan mengapa itu merupakan kesalahan.
- Tuliskan urutan langkah yang benar untuk menghindari data leakage. - Jelaskan dampak data leakage terhadap: (i) metrik evaluasi yang terlapor, dan (ii) performa model di dunia nyata (produksi).

Kesalahan

Leakage terjadi di Langkah 2: Fit TF-IDF vectorizer pada seluruh dataset (termasuk test) sebelum split.

Vectorizer "ngelihat" statistik test set pas ngitung IDF. Jadi info dari test "bocor" ke training → model dapet keuntungan yang nggak realistis. Pas deploy ke data beneran baru, performanya bisa drop.

Urutan yang Benar

- Load dataset
- Split dulu jadi train/test (misal 80:20)
- Fit vectorizer cuma di train
- Transform train & test pakai vectorizer yang sama (test nggak boleh fit ulang)
- Latih model di train
- Evaluasi di test

Prinsip: Semua "belajar" (fit) cuma dari train. Test cuma boleh "dipakai" (transform).

Dampak Leakage

(i) Metrik evaluasi: Terlapor terlalu bagus (akurasi/F1 inflated) → ilusi model sudah bagus

(ii) Performa produksi: Drop signifikan karena data baru beneran unseen → model nggak generalisasi baik

8. Evaluasi Model — Confusion Matrix 3 Kelas

Model klasifikasi sentimen 3-kelas menghasilkan confusion matrix berikut:

	Prediksi		
	Positif	Negatif	Netral
Aktual: Positif	45	5	
Aktual: Negatif	3	42	5
Aktual: Netral	2	8	40

- Hitung Precision, Recall, dan Fl -Score untuk setiap kelas (Positif, Negatif, Netral). Tunjukkan perhitungan lengkap.
- Hitung Macro-Average Fl dan Weighted-Average Fl .
- Kelas mana yang paling sulit diprediksi? Berikan analisis kualitatif berdasarkan nilai confusion matrix.

```
Perhitungan per Kelas

Positif:
      TP=45, FP=3+2=5, FN=5+0=5
      Precision = 45/50 = 0.90 | Recall = 45/50 = 0.90 | F1 = 0.90

Negatif:
      TP=42, FP=5+8=13, FN=3+5=8
      Precision = 42/55 ≈ 0.764 | Recall = 42/50 = 0.84 | F1 ≈ 0.80

Netral:
      TP=40, FP=0+5=5, FN=2+8=10
      Precision = 40/45 ≈ 0.889 | Recall = 40/50 = 0.80 | F1 ≈ 0.84

Macro & Weighted F1
      Macro-F1 = (0.90 + 0.80 + 0.84) / 3 ≈ 0.85
      Weighted-F1 = sama ≈ 0.85 (karena tiap kelas 50 sampel)
```

Paling Sulit
Negatif (F1 terendah: 0.80).

- 13 false positive → banyak dokumen lain dikira negatif
- 8 sampel Netral diprediksi Negatif → model bingung bedain nuansa "netral" vs "negatif"
- Kemungkinan overlap kosakata: kata kayak "biasa", "standar" bisa muncul di kedua kelas

9. Studi Kasus: Perbandingan Pipeline
Dua tim mahasiswa merancang pipeline klasifikasi teks untuk dataset ulasan produk e-commerce (2.000 dokumen, 3 kelas: Positif I Negatif / Netral, distribusi**50%/30**

```
TIM A: Lowercase Tokenisasi -i TF-IDF (semua fitur) -+ Naive Bayes
TIM B: Lowercase -i Hapus stopword —Y Stemming -+ TF—IDF (max
features=500)
Logistic Regression
```

- Identifikasi minimal 2 potensi masalah pada pipeline Tim A.
- Evaluasi kelebihan dan kelemahan pipeline Tim B.
- Dengan distribusi kelas 50%/30%/20%, metrik evaluasi apa yang lebih tepat digunakan: Accuracy atau Weighted F 1? Jelaskan alasan.
- Rancang pipeline versi perbaikan yang menggabungkan kelebihan keduanya.

Masalah Pipeline Tim A

Lowercase → Tokenisasi → TF-IDF(semua fitur) → Naive Bayes

- TF-IDF tanpa batasan fitur → vocabulary bisa ribuan, dimensi tinggi, risiko overfitting + komputasi berat
- Nggak ada stopword/stemming → kata berimbuhan ("membeli", "pembelian") dianggap beda fitur, padahal makna sama → informasi terpecah

Evaluasi Pipeline Tim B

Lowercase → Stopword → Stemming → TF-IDF(max_features=500) → Logistic Regression

Kelebihan:

- Preprocessing lebih lengkap → fitur lebih meaningful
- max_features=500 → kontrol dimensi, efisien
- Logistic Regression lebih robust daripada Naive Bayes untuk fitur berkorelasi

Kekurangan:

- Stemming Sastrawi kadang over-stem → bisa rusak makna
- max_features=500 mungkin terlalu sedikit buat nangkep nuansa ulasan
- Nggak disebut handling class imbalance (30%/20% minoritas)

Metrik yang Lebih Tepat: Weighted F1

Karena distribusi tdk seimbang (50/30/20), Accuracy bisa menyesatkan — model bisa dapat 50% accuracy cuma dengan nebak kelas mayoritas. Weighted F1 lebih adil karena pertimbangan performa tiap kelas sesuai proporsinya.

Pipeline Perbaikan (Gabungan Kelebihan)

```
1. Lowercase + Cleaning (hapus URL/emoji/karakter khusus)
2. Tokenisasi + Stopword Removal (daftar Bahasa Indonesia)
3. Normalisasi slang (ga→tidak, nyesel→menyesal)
4. Stemming selektif (Sastrawi + validasi manual untuk kata kritis)
5. TF-IDF: max_features=1000, min_df=2, max_df=0.8, ngram(1,2)
6. Logistic Regression + class_weight='balanced'
7. Evaluasi: Weighted F1 + Confusion Matrix
8. Validasi: Stratified 5-Fold CV
```

10. Pipeline Integratif End-to-End (Terstruktur)

Skenario: Anda diminta membangun sistem klasifikasi topik berita berbahasa Indonesia dengan 4 kelas (Politik, Ekonomi, Olahraga, Teknologi) dari 2.000 artikel. Lengkapi tabel rancangan pipeline berikut:

Komponen	Panduan Pengisian	Skor (Dosen)
(a) Preprocessing	Tahap apa saja? Mengapa diperlukan untuk data ini? Jawaban: `Lowercase → Cleaning (hapus HTML/URL) → Tokenisasi → Stopword Removal → Stemming (Sastrawi).` Berita Indo kaya imbuhan; stemming bantu satukan variasi kata ("perekonomian", "ekonomis" → "ekonomi") biar model nggak bingung.	
(b) Representasi Fitur	Pilih Bow, TF-IDF, atau lainnya. Justifikasi pilihan. Jawaban: TF-IDF + n-gram (1-2), max_features=1500. TF-IDF kasih bobot ke kata pembeda topik; bigram tangkap frasa penting kayak "suku bunga", "tim nasional" yang unigram nggak nangkep.	
(c) Algoritma Klasifikasi	Pilih 1 algoritma. Jelaskan alasannya. Jawaban: Logistic Regression (multinomial/softmax). Dataset 2.000 docs relatif kecil; LR cepat, interpretable, kerja bagus sama fitur sparse. Lebih robust daripada Naive Bayes yang asumsi independensi fitur sering dilanggar di teks.	

(d) Strategi Evaluasi	Pembagian data, metrik, handling imbalance. Jawaban: Split stratified 70:15:15 (train/val/test). Metrik utama: Weighted F1 + Confusion Matrix. Handling imbalance: class_weight='balanced'. 4 kelas dengan potensi distribusi nggak rata; stratified split jaga proporsi, weighted F1 lebih adil daripada accuracy.	
(e) Potensi Masalah & Solusi	Identifikasi 1 risiko utama & cara mengatasinya. Jawaban: Risiko: Overfitting karena vocabulary besar vs jumlah docs terbatas. Solusi: Batasi max_features, pakai regularisasi L2, validasi dengan 5-fold CV. Bahasa Indo morfologinya kaya → banyak variasi kata untuk konsep sama; kontrol kompleksitas model penting biar generalisasi baik.	

Catatan: Untuk setiap komponen, sertakan minimal 1 justifikasi berdasarkan karakteristik dataset (bahasa Indonesia, 4 kelas, 2.000 dokumen).